

Lab 2.1C Libraries, Linking, and EL Executable Structure

Pravina kumari Chaudhary

Bsc (Hons) Computer Science with Artificial Intelligence

Softwarica College of IT and E-commerce

ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi

4th october, 2026

TABLE OF CONTENT

1. Introduction	1
2. Practical C Program	1
2.1 Program Used	2
2.2 Observation	3
3. C Header Files	4
3.1 Observation	4
4. C Library Files	5
4.1 Observation	5
5. Static Linking	6
5.1 Observation	6
6. Dynamic Linking	7
6.1 Observation	7
7. Compare Static and Dynamic Executable Sizes	8
7.1 Observation	8
8. ELF Header	9
8.1 Observation	9
9. Program Headers	10
9.1 Observation	10
10. Dynamic Section	11
10.1 Observation	11
11. Checking Shared Libraries Using ldd	12
11.1 Observation	12
12. Static vs Dynamic Linking Comparison	13
13. Knowledge Check Questions	14
14. Conclusion	14
15. GitHub Repository	14

List of Figures

Figure1:SourceCode	1
Figure 2: Terminal execution and verification.....	2
Figure 3: C header files compilation.....	3
Figure 4: C library files compilation.....	4
Figure 5: Static linking compilation.....	5
Figure 6: Dynamic linking compilation.....	6
Figure 7: Size comparison.....	7
Figure 8: elf header compilation.....	8
Figure 9: Program headers compilation.....	9
Figure 10: Dynamic section compilation.....	9
Figure 11: ldd compilation.....	10

LIST OT TABLES

Table 1: Linking Comparison Table.....	11
---	-----------

1. Introduction

This is the lab report of Operating Systems, Security and Networks where we check how a programmer writes a C file that uses library functions like `printf()`, `getpid()`, and `time()`. When they compile it, many things in the background happen. The compiler changes the human-readable code to machine instructions. The linker then connects the program to external libraries. Finally, the OS reads the final binary file and loads it into the virtual memory (Bryant & O'Hallaron, 2015).

This lab is done to answer how a C program is linked to the C standard library, where do the library headers and compiled library code live on the disk, what is the difference between static and dynamic linking, and how can we inspect the resulting executable files.

2. A PRACTICAL C PROGRAM: PROCESS INFORMATION

2.1 Program Used

- I created a C program named **procinfo.c**.

The program uses functions such as **getpid()**, **getppid()**, **time()**, **localtime()**, and **printf()** to display information about the running process.



```
GNU nano 8.7.1 procinfo.c
#include <stdio.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    // Format and display process information
    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");

    return 0;
}
```

Figure 1 : Source Code

2.2 Observation

- I compiled and ran the program successfully. It displayed information such as the process ID, parent process ID, current time, and executable path.

```
pravina@Pravina:~/os-labs/LAB 2.1$ cat procinfo.c
#include <stdio.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    // Format and display process information
    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");

    return 0;
}
pravina@Pravina:~/os-labs/LAB 2.1$ gcc procinfo.c -o procinfo
pravina@Pravina:~/os-labs/LAB 2.1$ ./procinfo
=== Process Information ===
My Process ID (PID): 2481
Parent Process ID: 839
Current Time: Sun Oct  4 15:22:20 2026
Executable Path: /proc/self/exe
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 2: Terminal execution and verification

3. C Header Files

- I checked the system header files used by the C program:

```
ls /usr/include/stdio.h
ls /usr/include/unistd.h
ls /usr/include/time.h
```

3.1 Observation

- Header files contain the declarations needed by C programs. The system stores standard C header files mainly under **/usr/include/**.

```
pravina@Pravina:~/os-labs/LAB 2.1$ ls /usr/include
X11      complex.h  errno.h    ftw.h      inttypes.h  mcheck.h  neteconet  poll.h      resolv.h    spawn.h     sysexits.h  unistd.h
aio.h    cpio.h     error.h    fwctl      iproute2    memory.h  netinet    printf.h    rpc         stab.h      syslog.h    utime.h
aliases.h  crypt.h    execinfo.h  gawkapi.h  langinfo.h  misc      netipx     proc_service.h  rpcsvc     stdbit.h    tar.h      utmp.h
alloca.h  ctype.h    fcntl.h    gconv.h    lastlog.h   mntent.h  netiucv    protocols   sched.h     stdc-predef.h  termios.h  utmpx.h
ar.h      cxi        features-tine64.h  getopt.h    libgen.h    monetary.h  netpacket  pthread.h   scsi        stdint.h    tgmath.h   values.h
argp.h    dirent.h   features.h  glob.h      libintl.h   mqueue.h  netron     pty.h       search.h    stdio.h     thread_db.h  video
argz.h    dlfcn.h    fenv.h     gnu-versions.h  limits.h    mtd        netrose    pwd.h       semaphore.h  stdio_ext.h  threads.h   wait.h
arpa      drm        finclude   gnumake.h   link.h      net        nfs         rdma        setjmp.h    stdlib.h     time.h      wchar.h
asm-generic  elf.h     fntnsg.h   grp.h       linux       netash     nl_types.h  re_comp.h   sgty.h      string.h     ttyent.h    wctype.h
assert.h   endian.h   fnmatch.h  gshadow.h   locale.h    netatalk   nss.h       regex.h     shadow.h    strings.h    uchar.h     wordexp.h
byteswap.h  envz.h    fstab.h    iconv.h     malloc.h    netax25    obstack.h   regex.h     signal.h    sudo_plugin.h  ucontext.h  x86_64-linux-gnu
c++        err.h     fts.h      ifaddrs.h   math.h      netdb.h    paths.h     regulator   sound       syscall.h    ulimit.h    xen
pravina@Pravina:~/os-labs/LAB 2.1$
```

```
pravina@Pravina:~/os-labs/LAB 2.1$ ls /usr/include/stdio.h /usr/include/stdlib.h /usr/include/unistd.h /usr/include/time.h
/usr/include/stdio.h /usr/include/stdlib.h /usr/include/time.h /usr/include/unistd.h
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 3: C header files compilation

4. C Library Files

- I checked the available C library files on the system:

```
ls /usr/lib/aarch64-linux-gnu/libc.so*  
ls /usr/lib/aarch64-linux-gnu/libc.a
```

4.1 Observation

- The system contains shared library files such as **libc.so** and static library files such as **libc.a**. These libraries contain compiled code that programs can use.

A terminal window with a dark purple background and light green text. The user is at a prompt 'pravina@Pravina:~/os-labs/LAB 2.1\$'. They run 'grep "printf" /usr/include/stdio.h | head -2', which shows two lines of code from stdio.h. Then they run 'grep "getpid" /usr/include/unistd.h | head -2', which shows two lines of code from unistd.h. Finally, they run 'ls /usr/lib/x86_64-linux-gnu/libc.so*', which lists the shared library file 'libc.so.6'. They then run 'ls /usr/lib/x86_64-linux-gnu/libc.a', which lists the static library file 'libc.a'. The prompt returns after each command.

```
pravina@Pravina:~/os-labs/LAB 2.1$ grep "printf" /usr/include/stdio.h | head -2  
/* Maximum length of printf output for a NaN. */  
extern int fprintf (FILE *__restrict __stream,  
pravina@Pravina:~/os-labs/LAB 2.1$ grep "getpid" /usr/include/unistd.h | head -2  
extern __pid_t getpid (void) __THROW;  
pravina@Pravina:~/os-labs/LAB 2.1$ ls /usr/lib/x86_64-linux-gnu/libc.so*  
/usr/lib/x86_64-linux-gnu/libc.so  /usr/lib/x86_64-linux-gnu/libc.so.6  
pravina@Pravina:~/os-labs/LAB 2.1$ ls /usr/lib/x86_64-linux-gnu/libc.a  
/usr/lib/x86_64-linux-gnu/libc.a  
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 4: C library files compilation

5. Static Linking

- I created a statically linked version of the program using:

```
gcc -static procinfo.c -o procinfo_static
```

- Then I checked its size:

```
ls -lh procinfo_static
```

- I also checked its libraries:

```
ldd procinfo_static
```

5.1 Observation

- In static linking, the required library code is included inside the executable. Therefore, the executable is larger and ldd normally reports that it is not a dynamic executable.

```
pravina@Pravina:~/os-labs/LAB 2.1$ gcc -static procinfo.c -o procinfo_static
pravina@Pravina:~/os-labs/LAB 2.1$ ls -lh procinfo_static
-rwxr-xr-x 1 pravina pravina 961K Oct  4 16:13 procinfo_static
pravina@Pravina:~/os-labs/LAB 2.1$ ldd procinfo_static
        not a dynamic executable
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 5: Static linking compilation

6. Dynamic Linking

- I created a dynamically linked version using:

```
gcc procinfo.c -o procinfo_dynamic
```

- Then I checked its size:

```
ls -lh procinfo_dynamic
```

6.1 Observation

- In dynamic linking, the required libraries are not completely copied into the executable. They are loaded from shared library files when the program runs.

```
pravina@Pravina:~/os-labs/LAB 2.1$ gcc procinfo.c -o procinfo_dynamic
pravina@Pravina:~/os-labs/LAB 2.1$ ls -lh procinfo_dynamic procinfo_static
ls: cannot access 'procinfo_static': No such file or directory
-rwxr-xr-x 1 pravina pravina 16K Oct  4 16:02 procinfo_dynamic
pravina@Pravina:~/os-labs/LAB 2.1$ ldd procinfo_dynamic
        linux-vdso.so.1 (0x0000709391b6e000)
        libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x0000709391800000)
        /lib64/ld-linux-x86-64.so.2 (0x0000709391b70000)
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 6: Dynamic linking compilation

7. Compare Static and Dynamic Executable Sizes

- I compared the sizes of the static and dynamic executables:

ls -lh procinfo_static procinfo_dynamic

7.1 Observation

- The statically linked executable is much larger because library code is included inside it. The dynamically linked executable is smaller because it uses shared libraries.

```
pravina@Pravina:~/os-labs/LAB 2.1$ ls -lh procinfo_static procinfo_dynamic
-rwxr-xr-x 1 pravina pravina 16K Oct  4 16:10 procinfo_dynamic
-rwxr-xr-x 1 pravina pravina 961K Oct  4 16:13 procinfo_static
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 7: Size comparison

8. ELF Header

- I used **readelf** to inspect the ELF header:

readelf -h procinfo_dynamic

8.1 Observation

```
pravina@Pravina:~/os-labs/LAB 2.1$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x1140
  Start of program headers:              64 (bytes into file)
  Start of section headers:              14264 (bytes into file)
  Flags:                                  0x0
  Size of this header:                    64 (bytes)
  Size of program headers:                56 (bytes)
  Number of program headers:              14
  Size of section headers:                64 (bytes)
  Number of section headers:              31
  Section header string table index:      30
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 8: elf header compilation

9. Program Headers

- I checked the program headers using:

readelf -l procinfo_dynamic

9.1 Observation

- The program headers show how different parts of the executable are arranged in memory when the program is loaded. I could also see entries such as **LOAD** and **INTERP**.

```
pravina@Pravina:~/os-labs/LAB 2.1$ readelf -l procinfo_dynamic
Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
There are 14 program headers, starting at offset 64

Program Headers:
Type           Offset             VirtAddr           PhysAddr
               FileSiz            MemSiz             Flags             Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
               0x0000000000000310 0x0000000000000310 R                 0x8
INTERP         0x0000000000000374 0x0000000000000374 0x0000000000000374
               0x000000000000001c 0x000000000000001c R                 0x1
               [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x00000000000007b8 0x00000000000007b8 R                 0x1000
LOAD           0x0000000000000100 0x0000000000000100 0x0000000000000100
               0x0000000000000311 0x0000000000000311 R E              0x1000
LOAD           0x0000000000000200 0x0000000000000200 0x0000000000000200
               0x00000000000001c0 0x00000000000001c0 R                 0x1000
LOAD           0x0000000000000d80 0x0000000000000d80 0x0000000000000d80
               0x0000000000000290 0x0000000000000290 RW              0x1000
DYNAMIC        0x0000000000000d90 0x0000000000000d90 0x0000000000000d90
               0x00000000000001f0 0x00000000000001f0 RW                 0x8
NOTE           0x0000000000000350 0x0000000000000350 0x0000000000000350
               0x0000000000000024 0x0000000000000024 R                 0x4
NOTE           0x0000000000000217 0x0000000000000217 0x0000000000000217
               0x0000000000000030 0x0000000000000030 R                 0x8
NOTE           0x000000000000021a 0x000000000000021a 0x000000000000021a
               0x0000000000000020 0x0000000000000020 R                 0x4
GNU_PROPERTY   0x0000000000000217 0x0000000000000217 0x0000000000000217
               0x0000000000000030 0x0000000000000030 R                 0x8
GNU_EH_FRAME   0x0000000000000208 0x0000000000000208 0x0000000000000208
               0x0000000000000034 0x0000000000000034 R                 0x4
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000000 0x0000000000000000 RW              0x10
GNU_RELRO      0x0000000000000d80 0x0000000000000d80 0x0000000000000d80
               0x0000000000000280 0x0000000000000280 R                 0x1

Section to Segment mapping:
Segment Sections...
00
01      .interp
02      .note.gnu.build-id .interp .gnu.hash .dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt
03      .init .plt .plt.got .plt.sec .text .fini
04      .rodata .eh_frame_hdr .eh_frame .note.gnu.property .note.ABI-tag
05      .init_array .fini_array .dynamic .got .data .bss
06      .dynamic
07      .note.gnu.build-id
08      .note.gnu.property
09      .note.ABI-tag
10      .note.gnu.property
11      .eh_frame_hdr
12
13      .init_array .fini_array .dynamic .got
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 9: Dynamic section compilation

11. Checking Shared Libraries Using ldd

- I used:

ldd procinfo_dynamic

- I also checked the static executable:

ldd procinfo_static

It showed that the static executable is not a dynamic executable.

12.1 Observation

- The ldd command shows the shared libraries required by the dynamically linked program and the locations from which they are loaded.

```
pravina@Pravina:~/os-labs/LAB 2.1$ ldd procinfo_dynamic
linux-vdso.so.1 (0x00007140fa1eb000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007140f9e00000)
/lib64/ld-linux-x86-64.so.2 (0x00007140fa1ed000)
pravina@Pravina:~/os-labs/LAB 2.1$ ldd procinfo_static
not a dynamic executable
pravina@Pravina:~/os-labs/LAB 2.1$
```

Figure 10: ldd compilation

12. Static vs Dynamic Linking Comparison

Static Linking	Dynamic Linking
Library code is included in the executable.	Libraries are loaded separately.
Executable is larger.	Executable is smaller.
Does not depend on shared libraries at runtime.	Depends on shared libraries.
ldd normally reports it as not dynamic.	ldd shows required shared libraries.

Table 1: Linking Comparison Table

Knowledge Check Questions:

Question 1

What is the fundamental difference between a C header file and a shared library?

- A header file (.h) is a plain-text file that holds declarations such as function prototypes, types and macros. The compiler uses it at compile time, and it contains no executable code. A shared library (.so) is a compiled binary that holds the real machine code of the functions. The linker and the dynamic linker use it at link time and runtime. The header says what exists. The library is the working code.

Question 2

what is static and dynamic linking?

- Static linking copies the required library code from .a archives into the executable at build time. The result is self-contained but larger.

Dynamic linking stores only references to library functions in the executable. The dynamic linker loads the shared .so libraries into memory when the program starts. The result is a smaller executable, libraries shared between programs, and libraries that can be updated independently.

Question 3

What is ELF Header? Write down the commands to check headers.

- An ELF Header contains important information about a Linux executable, such as its type, architecture, and entry point.
- The command used in this lab is:

`readelf -h procinfo_dynamic`

- Other ELF information can be checked using:

**`readelf -l procinfo_dynamic`
`readelf -d procinfo_dynamic`**

Question 4

What does bookmark mean? Give an example.

- A bookmark is a saved reference or marker used to quickly return to a specific location or resource.

Example: A browser bookmark can save the URL of a frequently used website.

14. Conclusion

This lab taught me what happens after a C program is written: how it is compiled, how the standard library is linked to it, and how Linux loads it to run. By building a static and a dynamic version of the same program, I could see how linking changes the file size and its dependencies. The commands `gcc`, `readelf`, and `ldd` helped me check this directly on the executables. This gave me a clearer idea of how programs and libraries work together in Linux.

15. GitHub Repository

[Githhub Repository](#)

